Filter Selection from Pre-trained Convolutional Networks

COM31006 Computer Vision Assignment

Daniel Voice

Dataset link | Demonstration video link

1. Dataset

The Dataset consists of five classes: Bed, Desk, Lamp, Sofa, and Wardrobe. These classes were chosen as they are separable and common, so are present in the ImageNet dataset on which AlexNet was trained. Images are taken from royalty-free libraries Unsplash and Pexels. After manual cropping, I used a python script to resize the images to 128x128 pixels and enforce three (RGB) colour channels in *.jpg* format. Due to the small size of the dataset, an extensive set of transforms were applied to augment the dataset: RandomResizedCrop, RandomHorizontalFlip, RandomRotation, and ColorJitter. These augmentations add variation without the need for additional data, improving the model's ability to generalise, which reduces the risk of overfitting.

2. Part II — Fine-tuning AlexNet (A1)

2.1 Fine-tuning Setup

In order to conduct fine tuning I make the following changes to the AlexNet model:

- **Output Layer replacement** - The final classifier layer is replaced with a new fully connected layer that has the same input, but an output that corresponds to the number of classes in the dataset. This is essential to enable classification for my dataset.
- **Selective freezing** - The code first freezes the entire model, then unfreezes the classifier block. This means the feature extraction layers trained on ImageNet will not change, but the fully connected layers will learn to classify those same features into my classes.

With the above model changes in place, training is run with the following hyperparameters:

Hyperparameter	Value
Learning rate	2e-4 (0.0002)
Weight decay lambda	1e-3 (0.001)
Epochs	16
Batch size	8
Loss Function	CrossEntropyLoss
Optimiser	Adam

Figure 1 - Hyperparameters for A1 training

2.2 Reasoning

Learning rate, weight decay lambda and batch size were determined through manual experimentation, using GPU acceleration via Google Colab's free GPU access to speed up training. Adam was chosen as it tends to converge faster on small datasets compared to basic SGD. Epoch count was the last hyperparameter to be set, as the ideal stopping point was determined based on train vs test accuracy curves once other hyperparameters were decided.

A fixed learning rate is used, which is a weakness of my approach, but does make the process more simple and repeatable.
An important choice was the unfreezing of all three fully connected layers. I conducted initial testing by only unfreezing the final fully connected layer. After encountering issues with overfitting, I expanded to unfreeze all three classifier layers and increased dropout from the default 0.5 to 0.6 on the first two FC layers, in hopes of expanding the feature space and enabling more generalisation. Training the whole classifier is the approach I settled on, but without more testing it's difficult to determine which architecture choices would have been optimal.

2.3 Results

Once hyperparameters are decided, history for a real training cycle can be plotted.

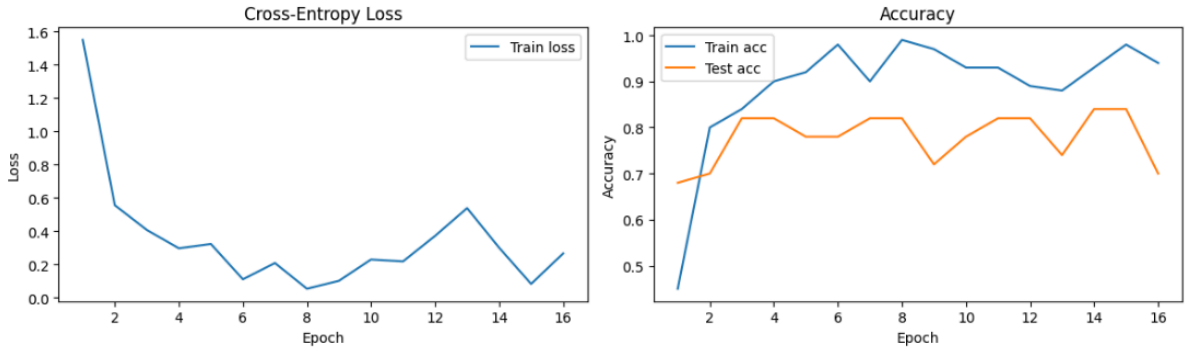


Figure 2 - A1 Training History

The training history for A1 shows reasonable performance, peaking at 82.5% accuracy. The decrease seen at epoch 16 doesn't appear consistently over training runs, but the trend of rarely breaking above low-80% accuracy does. AlexNet is an old architecture by modern standards, and an ensemble of 7 of them was used to achieve a top-5 error rate of 15.3% (84.7% accuracy) in the ImageNet recognition challenge in 2012. However, with a much smaller dataset, it's not unreasonable to conclude that A1 has the potential to achieve a better test accuracy than what's seen here, with the right setup.
The small dataset of 50 test images means a single misclassification can have a large effect on the result. Batch size of 8 for a set of 100 training images also gives only 12 gradient updates per epoch. This is quite a coarse training signal, causing instability.

3. Part III — Filter Comparison

3.1 Setup

AlexNet's Conv1 layer has 64 filters of shape (3, 11, 11). Classical filters are often represented as 3x3 or 5x5 filters. In order to enable a fair comparison, I wanted to scale up the classical filters to the same size as the learned ones. Gaussian and Sobel filters are mathematically defined, meaning I was able to construct them as 11x11 filters. The Laplacian filter is also mathematically defined, but larger kernels have to be built as a discrete approximation of second derivatives, making them inconsistent at larger kernel sizes. I therefore chose to use Laplacian-of-Gaussian as a replacement, enabling me to construct a simple, mathematically defined 11x11 filter for comparison.
In the original 2012 implementation of AlexNet, Conv1 consists of 96 filters. However, Torchvision's pretrained AlexNet model uses the "one weird trick" described in Krizhevsky's 2014 follow-up paper, meaning it has 64 filters in Conv1. Section 3 of my notebook starts by displaying all 64 filters (normalised 0-1 so negative values aren't lost). It then computes similarity metrics between the classical and learned filters.

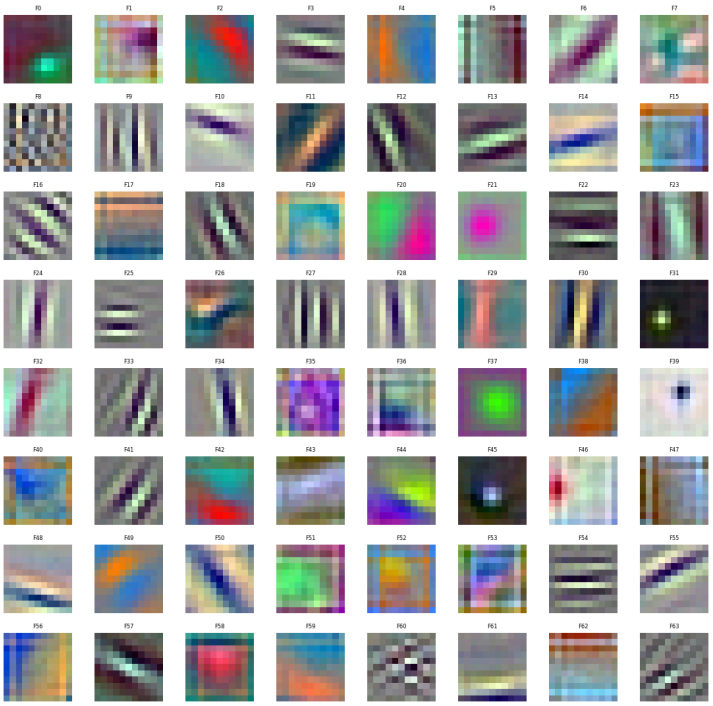


Figure 3 - All 64 Conv1 filters (RGB normalised weights)

3.2 Quantitative Similarity

I compute three complimentary metrics of similarity between each of the learned and classical filters: Pearson r (linear relationship), cosine similarity (angular alignment), and Frobenius distance (element wise euclidean distance). Pearson and cosine both capture structural similarity (shape of the filter pattern) while being somewhat robust to scale differences, whereas Frobenius captures raw numerical distance. Instead of displaying all 768 measurements, Pearson r is used as the primary metric. A Pearson correlation matrix shows the correlation between the four classical filters, and the 16 highest-matching learned filters.

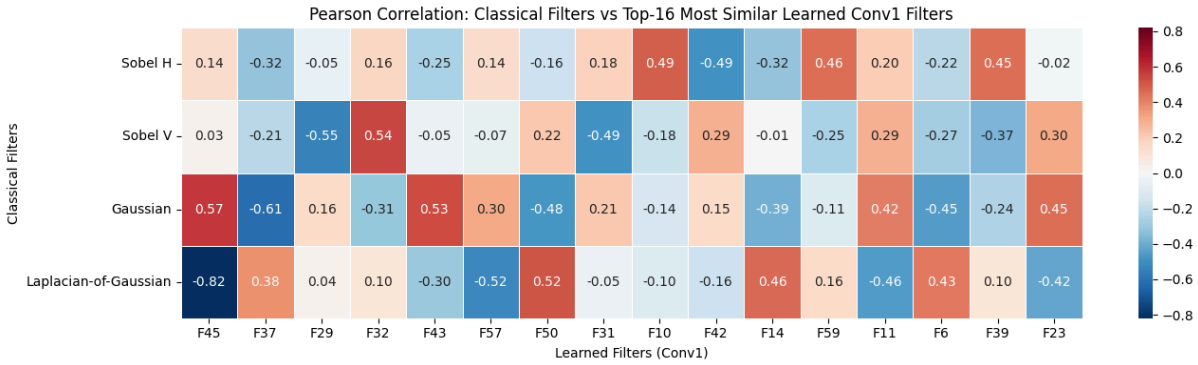


Figure 4 - Pearson Correlation Matrix. Shows only *Pearson Correlation Coefficient*.

The strongest finding from Figure 4 is r=-0.82 between the classical LoG and learned F45 filters, meaning F45 is close to an inverted LoG. This shows that Conv1 learned a filter that functions similarly to an edge detector. Correlations are notably weaker for Sobel Filters. For each classical filter, the learned filter with the highest Pearson Correlation Coefficient is chosen for comparison:

Classical	Filter	Pearson	Cosine	Frob
Sobel H	10	+0.490	+0.490	1.010
Sobel V	29	-0.549	-0.549	1.760
Gaussian	37	-0.613	-0.508	1.737
Laplacian-of-Gaussian	45	-0.820	-0.820	1.908

Figure 5 - Similarity measures for top matching filters

As shown in Figure 5, higher Frobenius distances are present where Pearson/cosine are more negative, confirming that some learned filters (29,37, 45) are inverted and similar to the classical filters, rather than copies. Pearson correlation and cosine similarity are equivalent for zero-mean filters (Sobel, LoG), but diverge for the Gaussian, whose non-zero mean introduces a bias that cosine similarity does not account for.

3.3 Qualitative Comparison

Once the closest matches to each classical filter are determined, the notebook produces two figures comparing the classical and learned filters, one showing how each filter looks, the other showing an example where the filters are passed over an example image from the dataset:

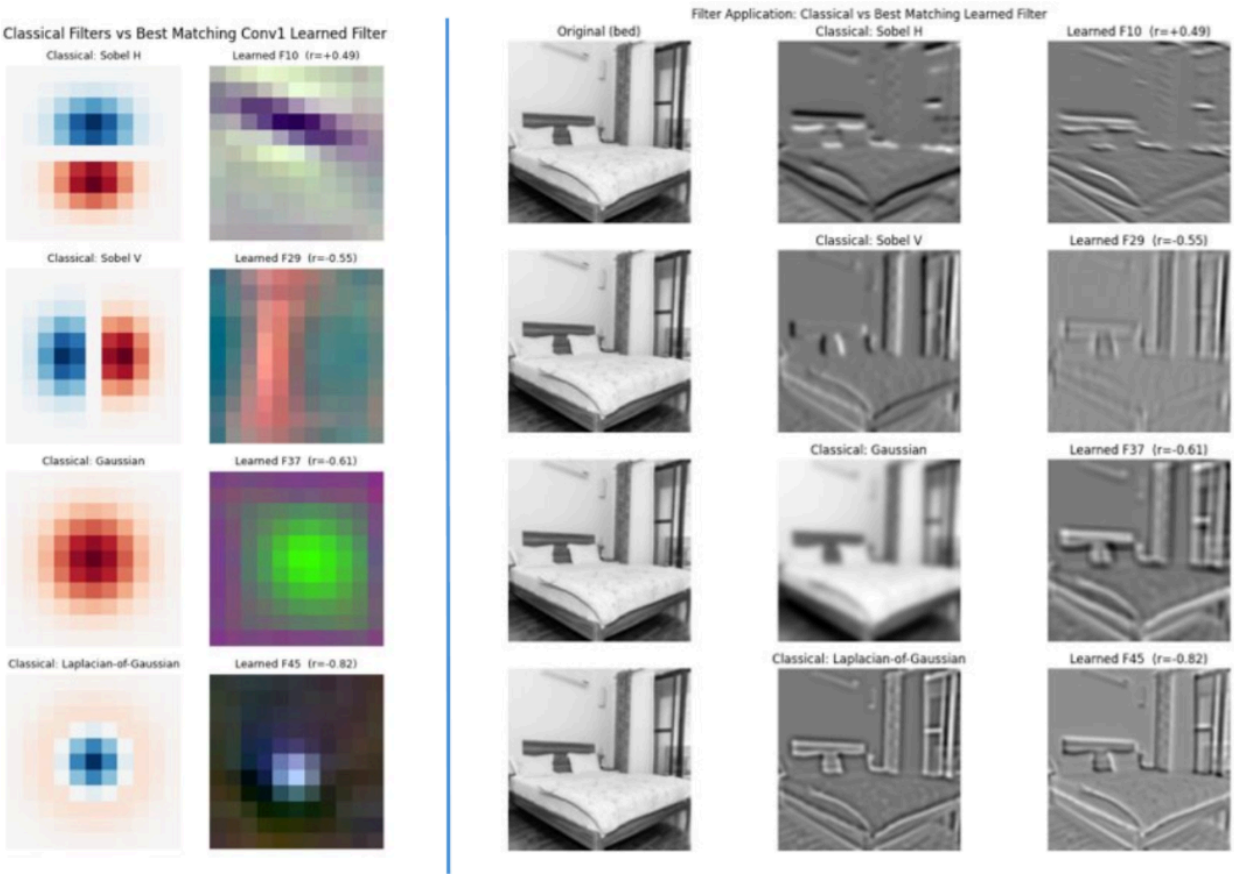


Figure 6 - Comparison of most similar filters

Qualitatively, you can see that there are similar patterns between the learned and classical filters. Laplacian and gaussian have closer matches due to their bloblike shape, whereas the learned equivalents to the Sobel filters are less neat, but still resemble directional edge detectors.
Filters at deeper layers will be less interpretable as they combine the outputs of previous layers and operate in deeper feature space, so filters can't be easily visualised.

4. Part IV — Novel Architecture A2 via Filter Selection

4.1 Selection Strategy & Justification

The L1 norm is a sum of the absolute weight of each three dimensions, providing one scalar value per filter that corresponds to how strong its activations are. L1 norm has been used as a metric for pruning in research, such as Xie et al (2019), who showed that for some models, many hidden neurons could be pruned based on L1 norm without harming predictive performance. L1 norm is also less sensitive to outlier weights compared to L2 Norm.
I chose to prune all but the top-K filters by L1 Norm from Conv2, where K is 128. This means a 33.3% reduction from 192 to 128 filters:

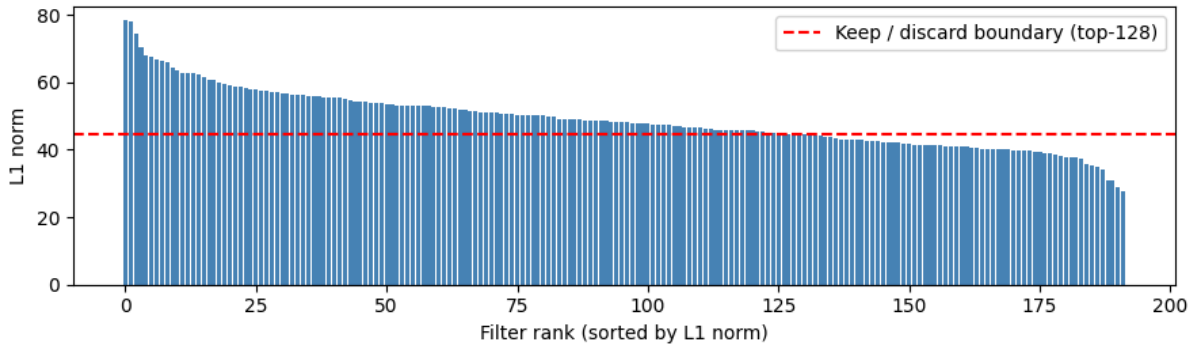


Figure 7 - Conv2 Filter L1 Norms for pretrained AlexNet model

Figure 7 shows a smooth decrease in L1 norm across filters. There is no clear boundary between “important” and “unimportant” filters, suggesting performance may be impacted by the pruning.

4.2 Architectural Adjustments

Simply zeroing the weights of chosen filters would not reduce the parameter count. I conduct structured pruning by replacing the old layer, without losing the weights of the retained neurons. This is achieved by storing the biases of the top-128 filters of Conv2, replacing the layer with a new convolutional layer with the same number of inputs, but 128 outputs, and applying the previous biases to the new layer.
The input channels of Conv3 directly correspond to the output channels of Conv2. Therefore, Conv3 now needs to be updated to prevent a crash on forward pass. The process is the same as for Conv2, but the replacement layer has the input channels updated, rather than outputs. The rest of the network doesn’t need to change, as Conv2’s input and Conv3’s output are unchanged.

4.3 Training A2

Conv3 was trained on all 192 inputs from Conv2. Removing >30% of them removes important context that Conv3 previously relied on, even if those contributions had lower weights. The A2 network should be retrained so the rest of the network can redistribute the load.
To train A2, the same hyperparameters as A1 are used (see Figure 1). However, in order to adapt to the pruned filters, more of the network is unfrozen. Conv1 and the pruned Conv2 remain frozen, but the remaining feature extraction layers Conv 3, 4, and 5 are unfrozen, along with the classifier. This is done because Conv3’s input channels have been reduced, so that layer onward needs to be retrained to adjust the weights.

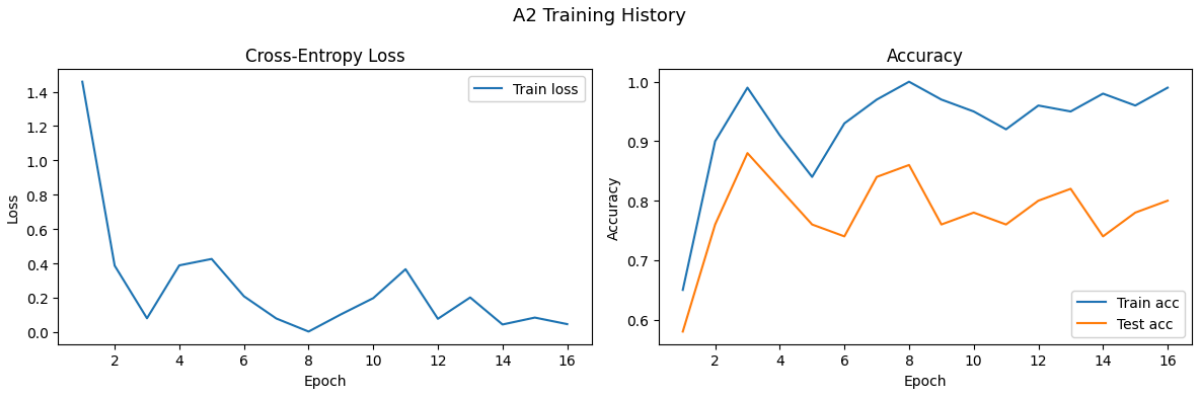


Figure 8 - A2 Training History

A2 shows less reliable performance than A1, but can reach higher peaks of accuracy. Where A1 rarely exceeded low-80% test accuracy, A2 can approach 90%.

5. Part V — Evaluation & Comparison

5.1 Parameter Comparison

As stated in Section 2.1, AlexNet has over 61M parameters, mostly in the fully connected layers. Since the convolutional layers account for so little of the parameters, pruning 33.3% of Conv2's filters only had a small effect on the parameter count of the models, reducing the total parameter count by 0.57% (-323,648).

5.2 Accuracy and Confusion Matrices

As well as full classification reports, the evaluation section provides data on accuracy.

Model	Accuracy	Weighted F1
A1	0.820	0.819
A2	0.760	0.762
Delta	-0.060	-0.057

Figure 9 - Test Accuracy, A1 vs A2, output from Part V

The data in Figure 9 shows that A2 performs marginally worse than A1 in testing. The -5.7% delta suggests an outsized performance reduction compared to the benefit of pruning 0.57% of the network's parameters using the current approach. It's possible that this could be improved with a change in approach.

Figure 10 shows the confusion matrices generated by Part V.

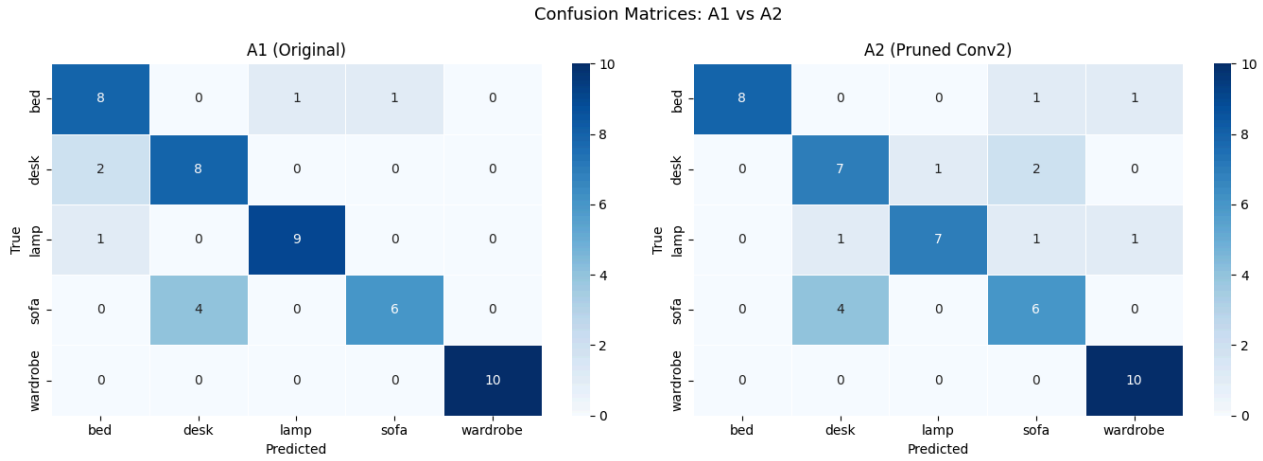


Figure 10 - Confusion Matrices for test set, A1 vs A2. Output by Part V.

The confusion matrices for A1 and A2 are similar, and show some patterns. Both models were able to identify wardrobes 100% of the time during testing, and both models have a tendency to falsely classify a sofa as a desk.

5.3 Visual Examples

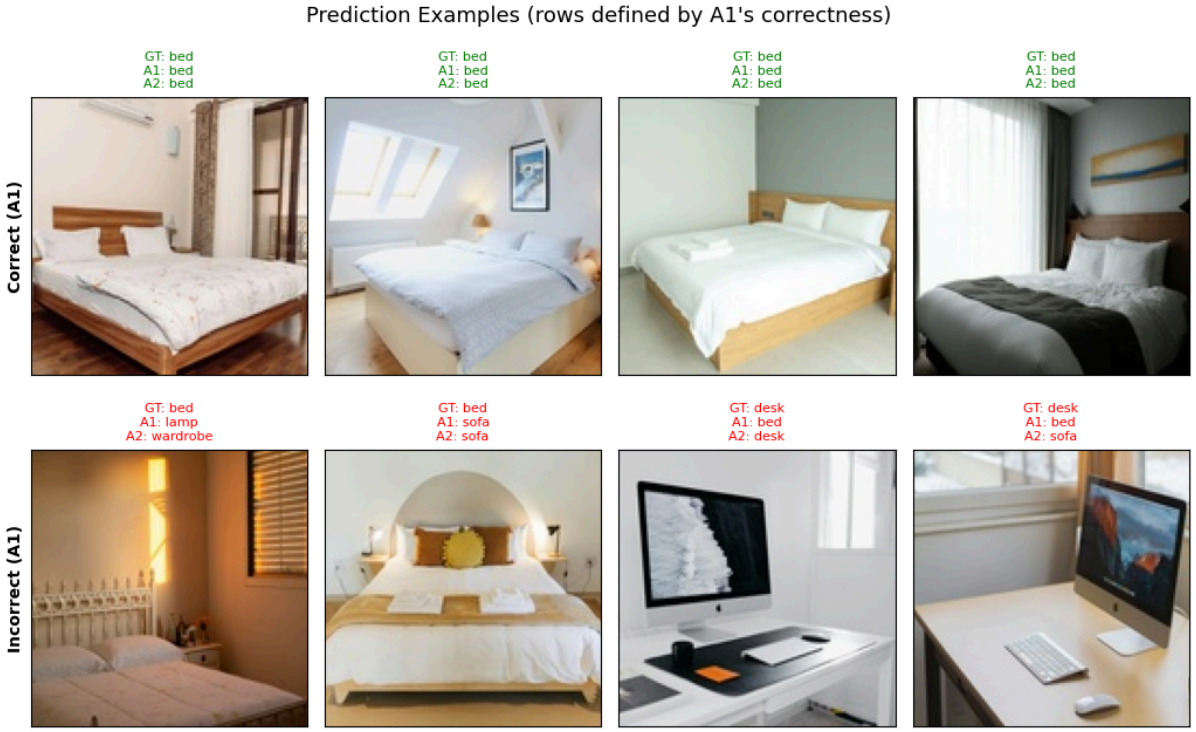


Figure 11 - Prediction Examples

The dataset contains some images that are more complex and challenging, some of which are shown in Figure 11, where one or both models fail to successfully classify the image. The dataset definitely leaves room for improved class and image selection, which could improve classification accuracy.

References

Xie X, Zhang H, Wang J, Chang Q, Wang J, Pal NR. Learning Optimized Structure of Neural Networks by Hidden Node Pruning With L_1 Regularization. IEEE Trans Cybern. 2020 Mar;50(3):1333-1346. doi: 10.1109/TCYB.2019.2950105. Epub 2019 Nov 25. PMID: 31765323.